ArquiSelect — Actualización del Diccionario de Predicados (Entregable 2)

Este documento complementa el diccionario del Entregable 1 con los predicados nuevos de `base_conocimiento.pl` para el **Módulo de Explicabilidad ("¿Por qué?")**, el **Módulo de Recomendación ("¿Qué falta?")** y la validación de entradas.

Modos de instanciación: `+` entrada enviada desde Python · `-` salida hacia Python · `?` puede usarse en ambos sentidos.

La columna **Desde Python** indica qué método de `PuenteProlog` (en `main.py`) llama al predicado. Los que dicen *Interno* solo los llama Prolog dentro del encadenamiento.

1. Hechos nuevos de la base de conocimiento

Hecho	Argumentos	Para qué sirve
`etiqueta/2`	`Requisito, Texto`	Texto legible de cada requisito. La interfaz arma sus casillas a partir de este hecho, así que un requisito nuevo aparece en pantalla sin modificar Python.
`pregunta/2`	`Requisito, Texto`	Pregunta que se le hace al usuario cuando ese dato es el que mejor desempata.
`excluyentes/2`	`RequisitoA, RequisitoB`	Requisitos que no pueden darse a la vez (equipo pequeño y equipo grande). Si ya se ingresó uno, el otro no se sugiere.
`umbral_desempate/1`	`Puntos` (2)	Diferencia mínima entre el primero y el segundo del ranking para que el resultado no se considere empate.
`peso_exclusion/1`	`Puntos` (5)	Castigo con el que se representa un descarte (`\+`) al medir el poder de desempate de un dato.
`max_sugerencias/1`	`N` (3)	Cuántos datos sugiere como máximo el módulo "¿Qué falta?".
`max_competidores/1`	`N` (3)	Cuántas hipótesis se comparan cuando falta evidencia y no hay empate.
`confianza_estado/2`	`Estado, Confianza`	Relaciona el estado del dictamen con un nivel de confianza: definitivo → alta, empate → media, insuficiente → baja, vacío → nula.

2. Validación de entradas (Nivel 1)

Firma	Modo	Descripción funcional	Desde Python
`validar_entrada/2`	`+Entrada, -Limpia`	Primera barrera contra datos mal formados. Si `Entrada` no es una lista cerrada (`is_list/1`) devuelve `[]`. Conserva solo los átomos sin variables (`ground/1`, `atom/1`) y los pasa a minúscula (`downcase_atom/2`). Descarta variables libres, números y términos compuestos.	Interno
`normalizar_lista/2`	`+Lista, -Normalizada`	Recorre la lista aplicando las comprobaciones anteriores. El corte separa la cláusula que acepta el elemento de la que lo ignora.	Interno
`preparar_requisitos/3`	`+Entrada, -Validos, -Desconocidos`	**Punto único de entrada de datos.** Valida una sola vez (`validar_entrada/2`), conserva los requisitos conocidos sin repetidos (`filtrar_validos/2`, `list_to_set/2`) y devuelve aparte los que no están en la base (`desconocidos/2`). Lo usan `diagnosticar_arquitectura/9` y `explicar_diagnostico/5`.	Interno
`preparar_requisitos/2`	`+Entrada, -Validos`	Versión corta de la anterior para cuando no hacen falta los desconocidos. La usa `que_falta/5`.	Interno

Cambio en un predicado existente: `diagnosticar_arquitectura/9` ahora prepara su entrada con `preparar_requisitos/3`, así que la validación queda en un solo sitio para los tres predicados principales. La firma y las salidas no cambiaron.

3. Módulo de Explicabilidad — "¿Por qué?"

Firma	Modo	Descripción funcional	Desde Python
`explicar_diagnostico/5`	`+Reqs, -Mejor, -Reglas, -Hechos, -Descartadas`	**Predicado principal del módulo.** Devuelve el estilo recomendado y la traza completa de auditoría: las reglas que se dispararon, los hechos comprobados y las hipótesis descartadas.	`explicar()`
`regla_disparada/3`	`+Validos, +Mejor, -Paso`	Cada solución es un paso de la traza con la forma `[Nivel, Regla, Detalle]`. Las cláusulas siguen el orden del encadenamiento (nivel 1 → 2 → 3), así que `findall/3` devuelve la traza ya ordenada. Un paso solo aparece si la meta que lo respalda tuvo éxito; por ejemplo, el paso de `estilo_viable/2 (\+)` aparece solo si la negación por fallo se cumplió.	Interno
`hechos_verdaderos/2`	`+Validos, -Hechos`	Evidencia ingresada que unificó con la base: `[Requisito, Grupo, AtributoQueExige]`.	Interno
`atributo_o_ninguno/2`	`+Requisito, -Atributo`	Atributo de calidad que exige el requisito. Si no exige ninguno devuelve `ninguno` (es el caso de `procesos_largos`, que solo excluye estilos).	Interno
`hipotesis_descartada/3`	`+Validos, +Mejor, -H`	Estilos que compartían parte de la evidencia pero quedaron fuera: `[Estilo, Tipo, Detalle, Comparte]`. El `Tipo` es `exclusion` cuando hay una condición excluyente (`incompatible/2`, es decir, `\+` en `estilo_viable/2`) y `sin_evidencia` cuando el estilo es viable pero su puntaje es ≤ 0. `Comparte` lista los atributos exigidos que el estilo sí cubría.	Interno
`relacionado_con_caso/3`	`+Estilo, +Validos, +Comparte`	Filtro para `sin_evidencia`: el estilo debe compartir algún atributo con el caso o haber sido penalizado por algún requisito. Así no se reportan estilos que no tuvieron nada que ver.	Interno
`detalle_primer_lugar/4`	`+Mejor, +Puntaje, +Resto, -Detalle`	Texto del paso `ranking/2` en la traza. Si otros estilos tienen el mismo puntaje que el primero, lo indica de forma explícita: en ese caso el orden lo decide `setof/3` (alfabético) y no la evidencia.	Interno

4. Módulo de Recomendación — "¿Qué falta?"

Firma	Modo	Descripción funcional	Desde Python
`que_falta/5`	`+Reqs, -Estado, -Confianza, -Competencia, -Sugerencias`	**Predicado principal del módulo.** Indica si la evidencia alcanza, qué hipótesis siguen en competencia y qué dato todavía no ingresado conviene preguntar.	`que_falta()`
`estado_dictamen/2`	`+Validos, -Estado`	`vacio` (no hay requisitos), `insuficiente` (un solo requisito o el mejor puntaje está por debajo del umbral de prioridad media), `empate` (el líder le saca menos de `umbral_desempate` al segundo) o `definitivo`. Los cortes hacen que se aplique la primera condición que se cumpla.	Interno
`mejor_puntaje/2`	`+Validos, -Puntaje`	Puntaje del primer estilo del ranking (0 si no hay ranking).	Interno
`margen_lider/2`	`+Validos, -Margen`	Diferencia de puntos entre el primero y el segundo del ranking (100 si hay un solo estilo viable).	Interno
`hipotesis_en_competencia/2`	`+Validos, -Competencia`	Estilos que siguen "parcialmente activos": si hay empate, los que están cerca del líder; si no, los	Interno

		mejores con puntaje positivo; si ninguno suma (lista vacía), todos los viables.	
`primeros/3`	`+N, +Lista, -Prefijo`	Los `N` primeros elementos de una lista, o menos si la lista es más corta.	Interno
`dato_candidato/2`	`+Validos, ?Requisito`	Requisito reconocido que aún no se ingresó y que no contradice lo ya ingresado. Usa doble negación por fallo, con `Requisito` ya instanciado por `requisito_categoria/2`.	Interno
`choca_con_ingresado/2`	`+Requisito, +Validos`	Verdadero si el requisito es excluyente con alguno ya ingresado (`excluyentes/2` en cualquier orden).	Interno
`efecto_dato/5`	`+Estilo, +Validos, +Requisito, -Delta, -Tipo`	Cuánto cambiaría el puntaje del estilo si se confirmara el requisito. `Tipo` puede ser `suma`, `resta`, `neutro` o `descarta`; en este último caso `Delta` es `-peso_exclusion`.	Interno
`tipo_cambio/2`	`+Delta, -Tipo`	Clasifica un cambio numérico en `suma`, `resta` o `neutro`.	Interno
`poder_desempate/5`	`+Competencia, +Validos, +Requisito, -Poder, -Efectos`	Peso del dato para separar las hipótesis. Con dos o más hipótesis es la distancia entre el efecto más favorable y el menos favorable; con una sola, el valor absoluto de su efecto. `Efectos` es `[[Estilo, Tipo, Delta], ...]`.	Interno
`poder_segun_cantidad/4`	`+N, +Max, +Min, -Poder`	Aplica la fórmula según cuántas hipótesis hay en competencia.	Interno
`sugerencias_desempate/3`	`+Validos, +Competencia, -Top`	Datos a preguntar, de mayor a menor poder: `[Requisito, Poder, Pregunta, Efectos]`. Usa `keysort/2`, que es estable, así que en un empate de poder se respeta el orden de la base. Solo sugiere datos con poder > 0.	Interno
`valores_de_pares/2`	`+Pares, -Valores`	Extrae los valores de una lista de pares `Clave-Valor`.	Interno

5. Consultas de apoyo para la interfaz

Firma	Modo	Descripción funcional	Desde Python
`requisitos_disponibles/1`	`-Lista`	`[[Requisito, Grupo, Etiqueta], ...]` con la que la interfaz dibuja las casillas, agrupadas por categoría.	`requisitos_disponibles()`
`estilos_disponibles/1`	`-Lista`	`[[Id, Nombre], ...]` con los nombres legibles de los estilos.	`estilos()`

6. Ejemplo de cómo se calcula el desempate

Con la entrada `[muchos_usuarios, carga_variable]`, serverless queda con 6 puntos y microservicios con 5. El margen es 1, menor que `umbral_desempate(2)`, así que el estado es **empate**. Para cada dato que falta, Prolog calcula su efecto en las dos hipótesis:

Dato candidato	Efecto en serverless	Efecto en microservicios	Poder
`equipo_pequeno`	+2	se descarta (−5)	2 − (−5) = **7**
`sin_experiencia_devops`	+2	se descarta (−5)	**7**
`procesos_largos`	se descarta (−5)	0	**5**
`presupuesto_bajo`	+2	−2	4

El sistema sugiere primero preguntar si el equipo tiene menos de 5 personas. Si la respuesta es sí, microservicios queda excluido y serverless pasa a 8 puntos con prioridad alta, y el dictamen queda **definitivo**.

7. Nota técnica sobre los predicados *steadfast*

`ranking/2` y `estado_dictamen/2` usan cortes, así que si se llaman con la salida ya instanciada (por ejemplo, `estado_dictamen(V, empate)`) Prolog puede saltarse las cláusulas que tienen el corte y dar una respuesta incorrecta: una lista vacía pasaría por empate. Por eso `margen_lider/2`, `mejor_puntaje/2` y `hipotesis_en_competencia/2` los consultan con una variable libre y comparan después con `=` o `==`.